SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43
STUDENT NAME: K. Ganesh
REGISTER NUMBER: 192411170

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS: **Total Marks: 50**

Code of Conduct:

I K. Ganesh / 192411170 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *K. Ganesh*

A3- Short Answer Test

Question 1 (5 Marks)
Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.
Question 2 (5 Marks)
Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.
Question 3 (5 Marks)
Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.
Question 4 (5 Marks)
Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.
Question 5 (5 Marks)
Responsive Layout: A college website must display properly on desktops, tablets, and mobile phones. How would you use CSS media queries and flexible layouts to achieve responsiveness?
Question 6(5 Marks)
CSS Styling: An online shopping website has plain product cards. How would you apply CSS colors, borders, shadows, and spacing to improve its visual appearance?
Question 7 (5 Marks)
Navigation Menu: A website navigation menu overlaps its content on smaller screens. How would you use CSS layout techniques to correct the problem?
Question 8 (5 Marks)
JavaScript Validation: A registration form accepts empty fields. How would you use JavaScript to validate the form before submission?

Question 9 (5 Marks)
Interactive Button: An e-learning website requires a button that displays a welcome message when clicked. How would you implement this functionality using JavaScript?
Question 104 (5 Marks)
. Dynamic Content: A shopping website needs to update the total price when a user changes the quantity of a product. How would you use JavaScript to calculate and display the updated price?

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1: Responsive Web Design

Concept and importance. Responsive web design (RWD) means building one website whose layout, images and text adjust automatically to the screen it is viewed on, from a small phone to a large desktop. Without it, users on phones must zoom and scroll sideways, and the company would need separate sites for each device. RWD is important because most visitors now use mobile devices, one codebase is cheaper to maintain, search engines (such as Google) rank mobile-friendly pages higher, and a comfortable experience on every device means more users stay and buy.

Key CSS techniques.

- **Viewport meta tag:** `<meta name="viewport" content="width=device-width, initial-scale=1">` makes the page use the real device width.
- **Fluid layouts:** percentages, `fr` units, `max-width` instead of fixed pixel widths.
- **Flexible images:** `img { max-width: 100%; height: auto; }`.
- **Media queries:** `@media (min-width: 768px) { ... }` apply different CSS at different screen sizes. A **mobile-first** approach writes phone styles first and adds queries for bigger screens.
- **Relative units:** `rem`, `em`, `vw` and `clamp()` for scalable text and spacing.

Flexbox or Grid for adaptive layouts. Flexbox arranges items along one axis and wraps them when space runs out (`flex-wrap: wrap`). Grid controls rows and columns together, and `repeat(auto-fit, minmax(220px, 1fr))` automatically changes the number of columns with the available width, without any media query.

Simple example layout (cards in 1, 2 or 3 columns, see Figure 1):

```
.cards { display: grid; gap: 16px; padding: 16px;
  grid-template-columns: 1fr; }              /* phone: 1 column */
@media (min-width: 600px) {
  .cards { grid-template-columns: repeat(2, 1fr); }   /* tablet */
}
@media (min-width: 1024px) {
  .cards { grid-template-columns: repeat(3, 1fr); }   /* desktop */
}
.cards img { max-width: 100%; height: auto; }
```

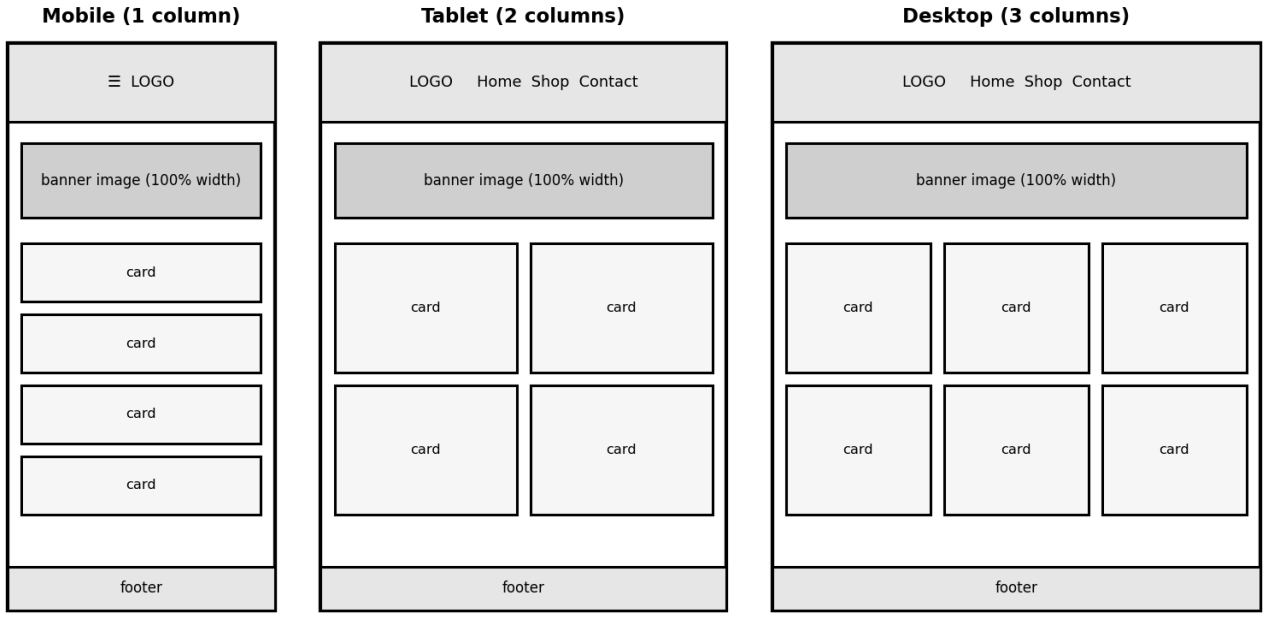


Figure 1: The same page on mobile, tablet and desktop

Improvements for user experience: use touch-friendly buttons (at least 44px), readable text (16px or more), compressed images with **`srcset`** and **`loading="lazy"`** for fast loading, a hamburger menu on phones, hide non-essential content on small screens, and test on real devices.

Question 2: Client-Side Form Validation

Concept and purpose. Client-side validation checks the data a user types in the browser, before the form is sent to the server. Its purpose is to give instant feedback (for example "Enter a valid email"), avoid a slow round trip for simple mistakes, reduce server load and guide users to correct their input. It improves usability but is not a security measure, because a user can bypass it, so the server must validate again.

JavaScript event handling in validation. Events trigger the checks: **`submit`** runs all checks and calls **`event.preventDefault()`** to stop the form if something is invalid, **`blur`** checks a field when the user leaves it, and **`input`** gives live feedback while typing. The handler reads **`input.value`**, tests it and shows a message in the page.

Role of regular expressions. A regular expression (regex) describes the pattern a value must match, and **`regex.test(value)`** returns true or false. For an email: **`^[^\s@]+@[^\s@]+\.[^\s@]{2,}$`** means some text, an @ sign, some text, a dot and at least two letters. For a 10-digit Indian mobile number: **`^[6-9]\d{9}$`** means it starts with 6 to 9 and has exactly 10 digits.

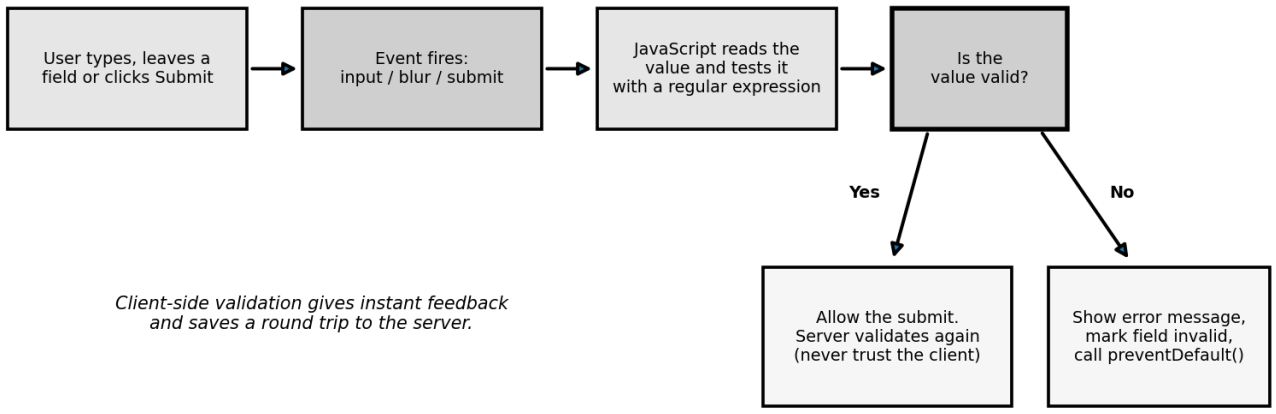


Figure 2: Validation flow in the browser

Example:

```
<form id="signup" novalidate>
  <label for="email">Email</label>
  <input type="email" id="email" name="email">
  <span id="emailError" role="alert"></span>
  <label for="phone">Phone (10 digits)</label>
  <input type="tel" id="phone" name="phone" inputmode="numeric">
  <span id="phoneError" role="alert"></span>
  <button type="submit">Submit</button>
</form>
```

```
const emailRe = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
const phoneRe = /^[6-9]\d{9}$/;

function check(input, re, message, out) {
  const ok = re.test(input.value.trim());
  out.textContent = ok ? "" : message;
  input.setAttribute("aria-invalid", String(!ok));
  return ok;
}

const email = document.getElementById("email");
const phone = document.getElementById("phone");
const eErr = document.getElementById("emailError");
const pErr = document.getElementById("phoneError");

email.addEventListener("blur", () =>
  check(email, emailRe, "Enter a valid email address", eErr));
phone.addEventListener("blur", () =>
  check(phone, phoneRe, "Enter a 10-digit mobile number", pErr));

document.getElementById("signup").addEventListener("submit", (e) => {
  const a = check(email, emailRe, "Enter a valid email address", eErr);
  const b = check(phone, phoneRe, "Enter a 10-digit mobile number", pErr);
  if (!(a && b)) e.preventDefault();      /* stop sending bad data */
});
```

Improvements: show clear messages next to each field with **`aria-live`**/**`role="alert"`** and **`aria-invalid`**; also add HTML5 attributes (**`required`**, **`type="email"`**, **`pattern`**) as a fallback; keep regex simple so valid addresses are not rejected; **always validate and sanitize again on the server**, escape output to prevent XSS, use parameterised queries, HTTPS and CSRF tokens; and avoid revealing too much in error messages.

Question 3: CSS Box Model, Positioning and Layout Breaking

CSS box model. Every HTML element is a rectangular box made of four parts, from inside to outside: **content** (text or image), **padding** (space between content and border), **border** (the line around the padding) and **margin** (transparent space outside the border that separates it from other elements). By default, **`width`** only sets the content, so padding and border are added on top and the box becomes wider than expected. Setting **`box-sizing: border-box`** makes the width include padding and border, which is much easier to control.

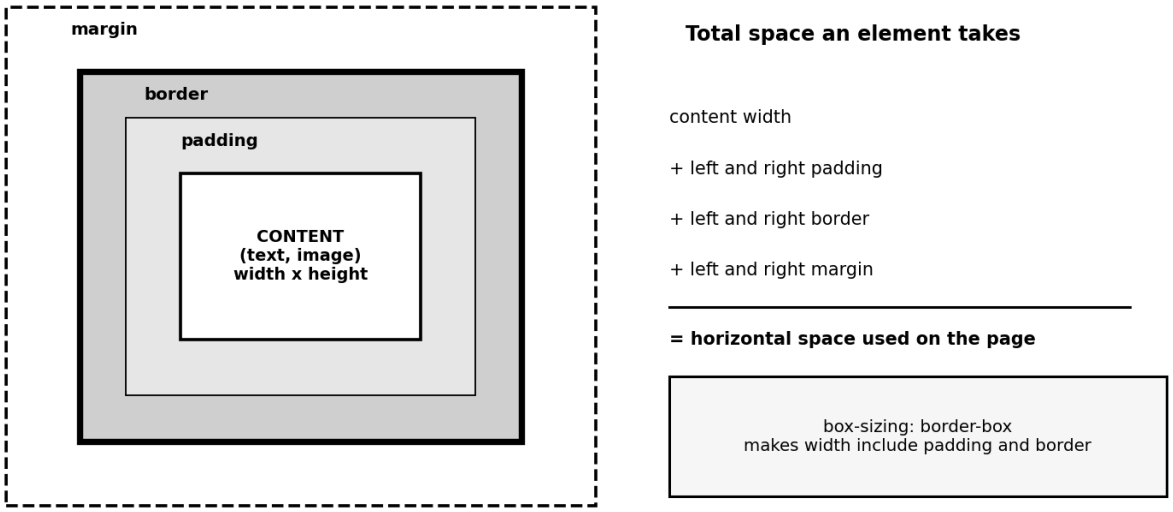


Figure 3: The CSS box model

CSS positioning and its effect on layout.

Value	Behaviour	Effect on layout
`static`	Default; normal document flow	Elements follow one another in order
`relative`	Moves from its normal place using top/left; its original space is kept	Small shifts; also the anchor for absolute children
`absolute`	Placed relative to the nearest positioned ancestor	Removed from the flow, so other elements move up and can overlap it
`fixed`	Fixed to the browser window while scrolling	Removed from the flow; can cover content
`sticky`	Normal flow until a scroll point, then sticks	Stays in the flow, useful for headers

Possible causes of layout breaking when the window is resized:

- Fixed pixel widths (`width: 960px`) that are larger than a small screen, causing horizontal scrolling.
- Absolutely or fixed positioned elements that are taken out of the flow and overlap other content.
- Missing `box-sizing: border-box`, so padding and border make boxes wider than their container.
- Images without `max-width: 100%`, uncleared floats, long unbroken text, and a missing viewport meta tag.

How media queries resolve this. Media queries apply different rules at a chosen width, so the layout changes before it breaks:

```
.wrapper { width: 100%; max-width: 1100px; margin: 0 auto; }
.sidebar, .content { float: none; width: 100%; }  /* phone: stacked */
@media (min-width: 768px) {
  .wrapper { display: flex; gap: 20px; }
  .sidebar { flex: 0 0 240px; }
  .content { flex: 1; min-width: 0; }
}
```

Best practices for a stable, responsive layout: use `box-sizing: border-box` everywhere, prefer flexible units (%, `fr`, `rem`) and `max-width`, use Flexbox or Grid instead of floats and heavy absolute positioning, set `img { max-width: 100%; }`, add the viewport tag, design mobile-first, add `overflow-wrap: anywhere` for long text, and test at many widths.

Question 4: DOM and Dynamic Content Updates

Document Object Model (DOM). The DOM is the browser's tree-shaped, in-memory representation of an HTML page. Every element, attribute and piece of text becomes a node. The DOM is the interface that lets JavaScript read and change the page, and CSS styles are applied to it. Its role in web development is to make pages dynamic: content can change, appear or disappear in response to user actions, without reloading the page.

How JavaScript manipulates the DOM. Typical steps are: select a node (`getElementById`, `querySelector`), change it (`textContent`, `classList`, `setAttribute`, `style`), create new nodes (`createElement`) and add or remove them (`appendChild`, `remove()`). Events such as click trigger these changes, and `fetch()` can load fresh data from the server in the background (AJAX) and display it in the page.

Example (adds an item to a cart list and updates the count):

```
<h2 id="status">Items in cart: 0</h2>
<ul id="list"></ul>
<button id="add">Add item</button>
```

```
let count = 0;
document.getElementById("add").addEventListener("click", () => {
  count++;
  const li = document.createElement("li");
  li.textContent = "Item " + count;
  document.getElementById("list").appendChild(li);
  document.getElementById("status").textContent =
    "Items in cart: " + count;
});
```

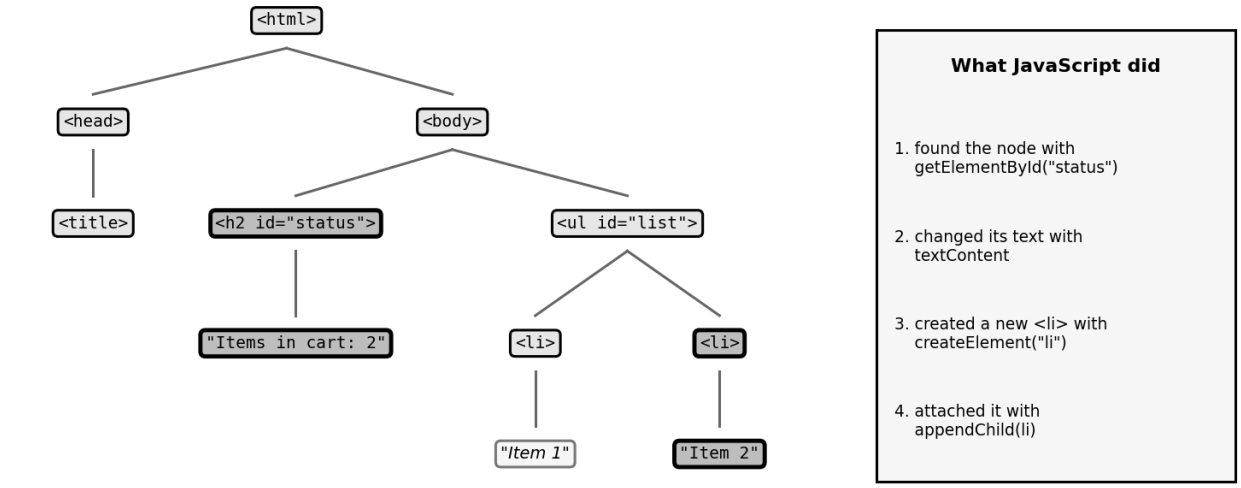


Figure 4: DOM tree and the nodes changed by the script

Loading data from a server without reloading the page works the same way:

```
fetch("/api/products?category=shoes")
  .then((res) => res.json())
  .then((items) => {
    const frag = document.createDocumentFragment();
    items.forEach((p) => {
      const li = document.createElement("li");
      li.textContent = p.name + " - Rs. " + p.price;
      frag.appendChild(li);
    });
    document.getElementById("list").replaceChildren(frag);
  })
  .catch(() => alert("Could not load products"));
```

Improvements for performance and user experience: add many nodes at once with a DocumentFragment (one reflow instead of many), use event delegation (one listener on the parent), prefer **`textContent`** over **`innerHTML`** to avoid XSS, debounce frequent events, show a loading indicator and an error message for failed requests, and announce updates to screen readers with **`aria-live="polite"`**.

Question 5: Responsive Layout for a College Website

I would build the page mobile-first with a flexible layout and add media queries at the points where the design needs to change.

- **Viewport and fluid base:** add the viewport meta tag, use **`box-sizing: border-box`**, relative widths (**`%`**, **`fr`**) and **`img { max-width: 100%; }`**.

- **Flexible layout:** Flexbox for the header and navigation, Grid for the content sections (About, Courses, News).
- **Media queries:** phones below 600px show one column with a hamburger menu, tablets from 600px use two columns, and desktops from 1024px use three columns with the menu in one row.

```
.page { display: grid; gap: 16px; padding: 16px;
  grid-template-columns: 1fr; }                /* phone */
.nav ul { display: none; }                      /* hamburger instead */
@media (min-width: 600px) {
  .page { grid-template-columns: repeat(2, 1fr); }    /* tablet */
}
@media (min-width: 1024px) {
  .page { grid-template-columns: repeat(3, 1fr);
          max-width: 1200px; margin: 0 auto; }
  .nav ul { display: flex; gap: 20px; }
}
```

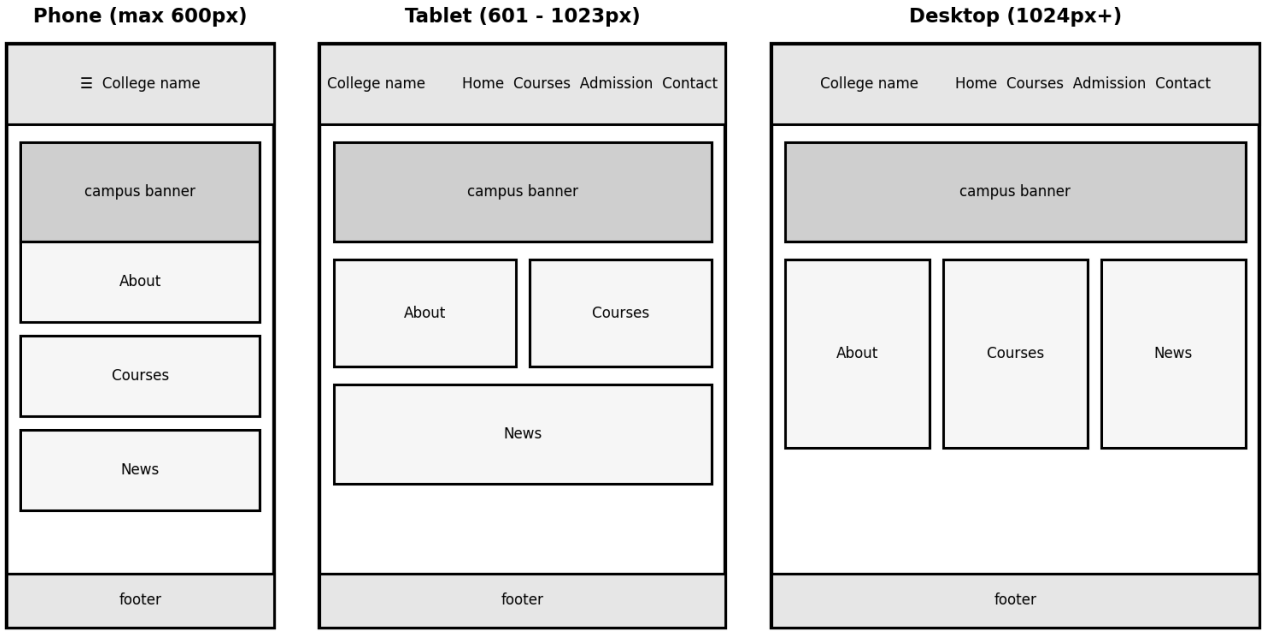


Figure 5: College website layout on phone, tablet and desktop

Question 6: CSS Styling of Product Cards

I would turn the plain card into a clear, clickable-looking card step by step:

- **Colours:** a white card on a light-grey page background, dark text for the product name, and one accent colour for the price and button. Keep contrast high enough to read.
- **Borders:** a thin light border with **`border-radius`** for soft corners.
- **Shadows:** a subtle **`box-shadow`** to lift the card off the page, slightly stronger on hover.
- **Spacing:** inner **`padding`** so content does not touch the edges, a **`margin`** or grid **`gap`** between cards, and consistent spacing between image, title, price and button.

```
body { background: #f3f3f3; }
.product-card {
  background: #fff;
  border: 1px solid #ccc;
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
  padding: 16px;
  margin: 12px;
  transition: box-shadow 0.3s, transform 0.3s;
}
.product-card:hover {
  box-shadow: 0 8px 18px rgba(0, 0, 0, 0.25);
  transform: translateY(-4px);
}
.product-card h3 { margin: 12px 0 4px; color: #111; }
.product-card .price { font-weight: bold; color: #b12704; }
.product-card button { margin-top: 10px; padding: 10px 16px; border: 0;
  border-radius: 6px; background: #222; color: #fff; cursor: pointer; }
```

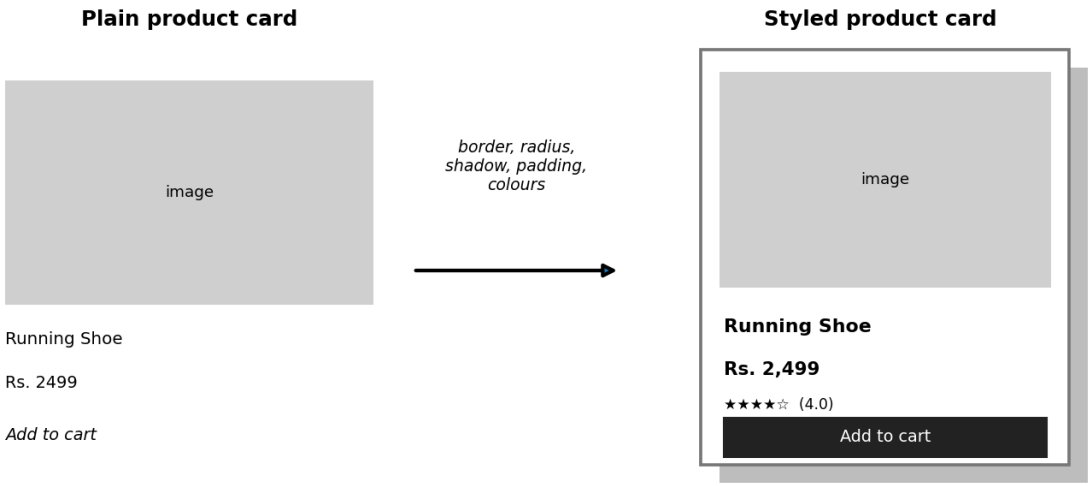


Figure 6: Plain card compared with the styled card

Question 7: Navigation Menu Overlapping Content

Cause: the menu is usually **`position: fixed`** or **`absolute`** (so it is removed from the normal flow and no space is kept for it), or it has fixed widths and many links that do not fit on a small screen. The page content then starts at the top and sits underneath the menu.

Correction using CSS layout techniques:

- Keep the menu in the normal flow, or use **`position: sticky`**, which still reserves its space. If it must be fixed, add **`padding-top`** to the body equal to the menu height.
- Use Flexbox with **`flex-wrap: wrap`** so links wrap instead of overflowing, and use relative widths.
- On small screens, collapse the links into a hamburger menu that opens as a block under the header, pushing the content down.

```
.site-header { position: sticky; top: 0; z-index: 100;
  display: flex; flex-wrap: wrap; align-items: center;
  justify-content: space-between; background: #fff; }
.menu { display: none; width: 100%; }          /* phone: collapsed */
.menu.open { display: block; }                 /* pushes content down */
.menu a { display: block; padding: 12px; }
@media (min-width: 768px) {
  .menu { display: flex; width: auto; gap: 16px; }
  .toggle { display: none; }
}
```

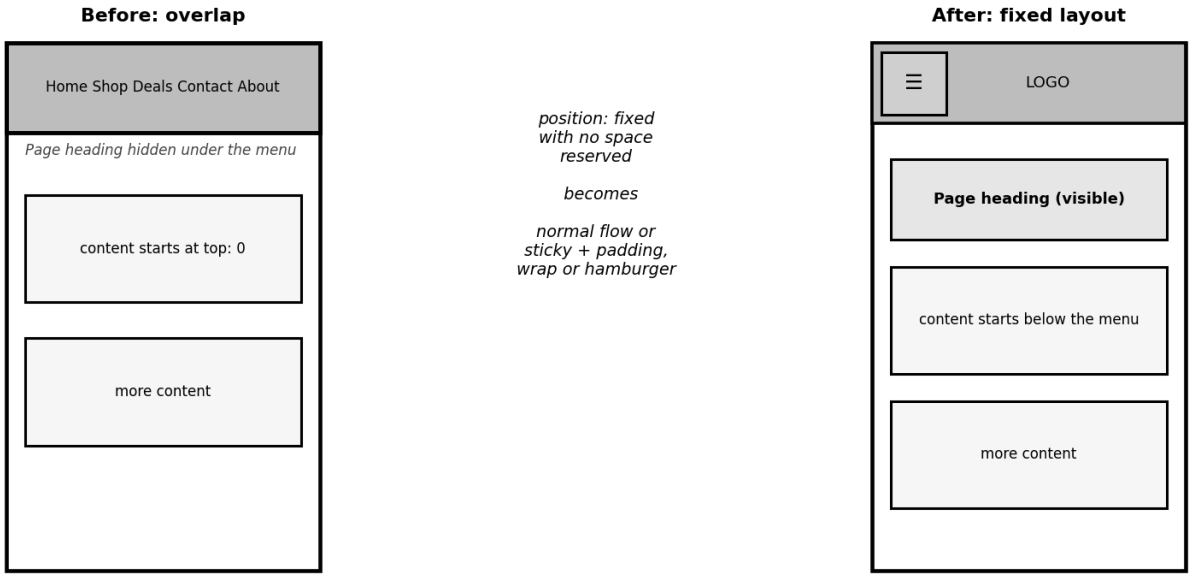


Figure 7: Menu overlapping the content, and the corrected layout

Question 8: JavaScript Validation of a Registration Form

I would listen for the form's **`submit`** event, check that every required field is not empty (after **`trim()`**), show a message for each empty field and call **`event.preventDefault()`** so the form is not sent until everything is filled.

```
<form id="regForm" novalidate>
  <label for="name">Name</label>
  <input type="text" id="name" name="name">
  <span id="nameError" role="alert"></span>
  <label for="email">Email</label>
  <input type="email" id="email" name="email">
  <span id="emailError" role="alert"></span>
  <label for="password">Password</label>
  <input type="password" id="password" name="password">
  <span id="passwordError" role="alert"></span>
  <button type="submit">Register</button>
</form>
```

```
document.getElementById("regForm").addEventListener("submit", (e) => {
  let valid = true;
  ["name", "email", "password"].forEach((id) => {
    const input = document.getElementById(id);
    const error = document.getElementById(id + "Error");
    if (input.value.trim() === "") {
      error.textContent = "This field is required";
      valid = false;
    } else {
      error.textContent = "";
    }
  });
  if (!valid) e.preventDefault();   /* block empty submission */
});
```

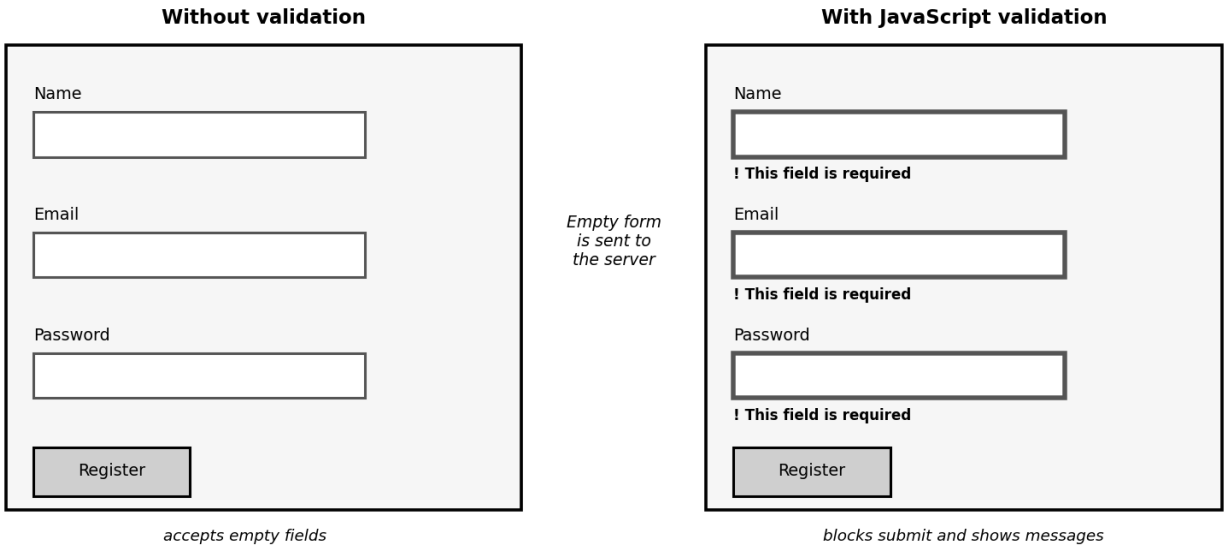


Figure 8: Registration form without and with JavaScript validation

The same code can later be extended with regular expressions for email and password strength, and the server must repeat the checks.

Question 9: Interactive Welcome Button

I would give the button an id, select it with JavaScript, attach a **`click`** event listener, and in the handler write the welcome message into a paragraph (or show it with **`alert()`**).

```
<button id="welcomeBtn">Start Learning</button>
<p id="msg" aria-live="polite"></p>
```

```
const btn = document.getElementById("welcomeBtn");
const msg = document.getElementById("msg");

btn.addEventListener("click", () => {
  msg.textContent = "Welcome to our e-learning platform! Happy learning.";
});
```

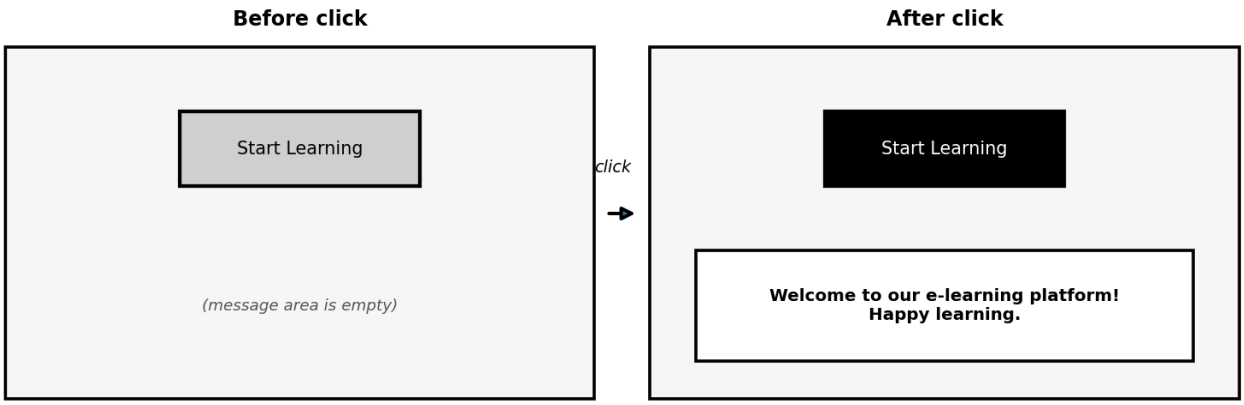


Figure 9: The message appears when the button is clicked

Using **`addEventListener`** keeps HTML and JavaScript separate, and **`aria-live="polite"`** makes screen readers read the message when it appears.

Question 10: Updating the Total Price Dynamically

I would store the unit price, listen for the **`input`** event on the quantity field, calculate **`price x quantity`** and write the result into the total element. The function also runs once at page load so the first total is correct.

```
<p>Smart Watch - Price: Rs. <span id="price">500</span></p>
<label for="qty">Quantity</label>
<input type="number" id="qty" value="1" min="1" max="10">
<p>Total: <strong id="total">Rs. 500</strong></p>
```

```
const price = Number(document.getElementById("price").textContent);
const qty = document.getElementById("qty");
const total = document.getElementById("total");

function updateTotal() {
  let q = parseInt(qty.value, 10);
  if (isNaN(q) || q < 1) q = 1;           /* guard against bad input */
  total.textContent = "Rs. " + (price * q).toLocaleString("en-IN");
}

qty.addEventListener("input", updateTotal);
updateTotal();
```

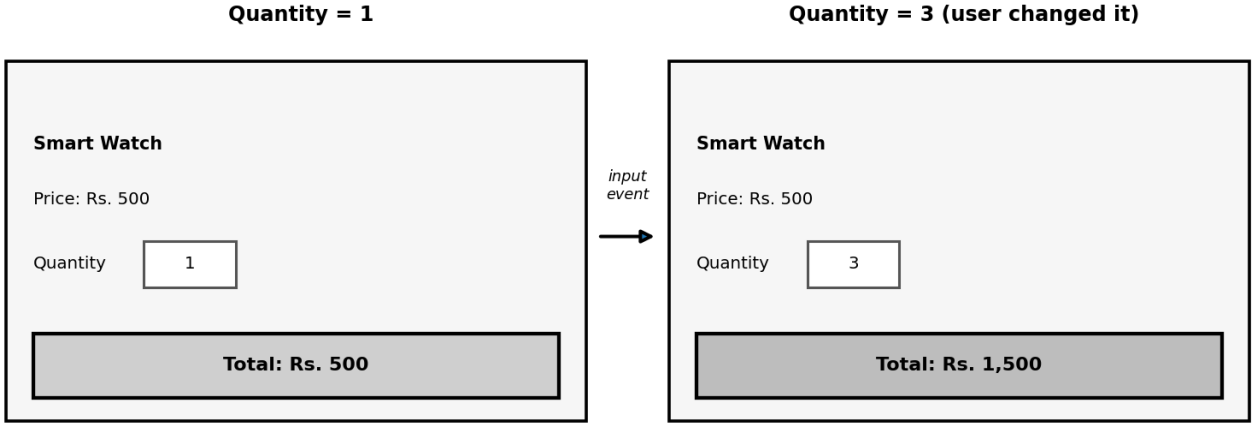


Figure 10: Total price changes when the quantity changes

Example: with a price of Rs. 500, changing the quantity from 1 to 3 changes the total from Rs. 500 to Rs. 1,500 immediately, without reloading the page. The price should also be calculated and checked on the server at checkout.